

Frequency Response & Polar Diagrams

Table of Contents

Introduction - Polar Diagrams of the Frequency Response.....	1
With a Little Help from the 'help' Command!.....	1
Remark.....	4
Polar Diagrams of the Frequency Response using MATLAB.....	5
Example 1: A Strictly Proper Transfer Function with Real Poles	5
Example 2: A Couple of Complex Poles, varying the Damping Factor.....	6
Example 3: a Minimum-Phase LTI System with a Pole at the Origin.....	7

Introduction - Polar Diagrams of the Frequency Response

Consider an **asymptotically stable** LTI continuous-time that is **fully described** (that is, no common factors, i.e. no hidden dynamics) by the transfer function $G(s)$



Moreover, consider $u(t) = \left[A \sin(\bar{\omega} t) \right] \cdot 1(t) \implies \mathcal{L}(u(t)) = U(s) = \frac{A \bar{\omega}}{s^2 + \bar{\omega}^2}$

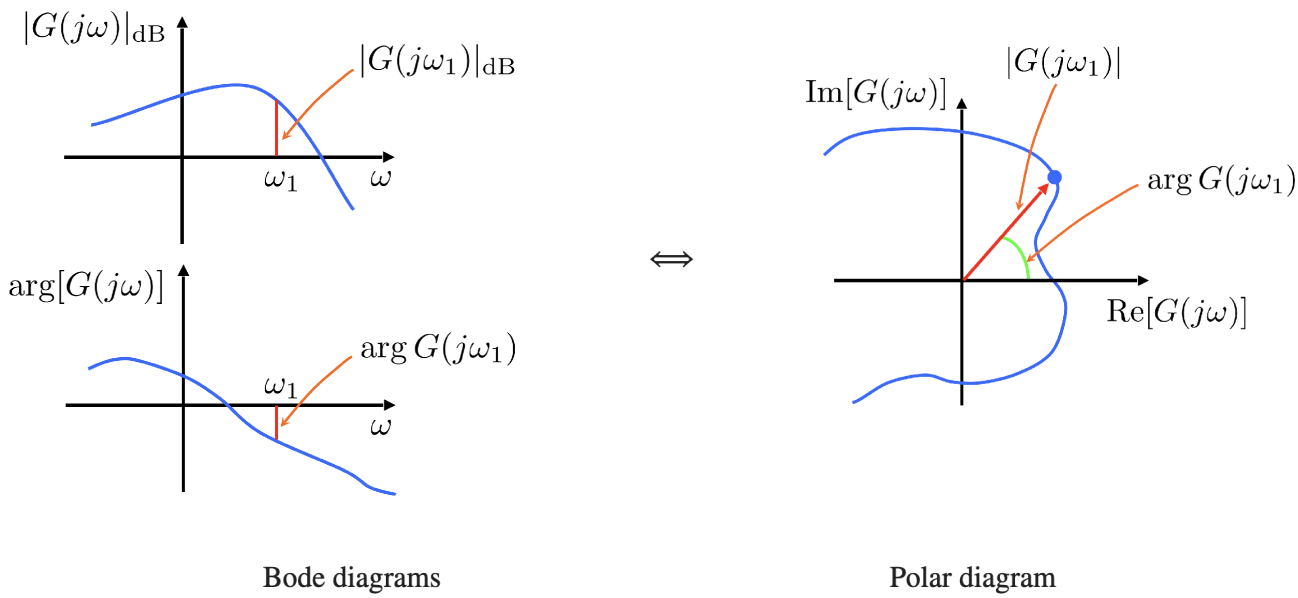
The regime sinusoidal response (approx. equal to $y(t)$ for $t > t_s$, where t_s is the settling time of the step response of the system $G(s)$) is:

$$y_{\text{reg.}}(t) = \left[|G(j \bar{\omega})| \cdot A \sin(\bar{\omega} t + \bar{\varphi}) \right] 1(t), \quad \bar{\varphi} = \arg G(j \bar{\omega})$$

The **frequency response** of the system is defined as: $G(j \omega), \quad \omega \geq 0$

Polar and Bode diagrams are related:

$$G(j \omega) = |G(j \omega)| e^{j \arg G(j \omega)} = \text{Re}[G(j \omega)] + j \text{Im}[G(j \omega)]$$



With a Little Help from the 'help' Command!

Here is the documentation related to the Control System Toolbox function we will use in this live script:

```
help evalfr

--- help for DynamicSystem/evalfr ---

DynamicSystem/evalfr - Evaluate system response at specific frequency
evalfr is a simplified version of freqresp meant for quick evaluation of
the system response at any point in the complex plane.

Syntax
frsp = evalfr(sys,x)

Input Arguments
sys - Dynamic system
      dynamic system model | model array
```

x – Point in complex plane
complex scalar

Output Arguments

frsp – Frequency response
complex scalar | complex array

Examples

Evaluate Discrete-Time Transfer Function
Evaluate Frequency Response of Identified Model at Given Frequency
Frequency Response of MIMO State-Space Model

See also `bode`, `freqresp`, `sigma`

Introduced in Control System Toolbox before R2006a
Documentation for `DynamicSystem/evalfr`

help `freqresp`

freqresp – Evaluate system response over a grid of frequencies
Use `freqresp` to evaluate the system response over a grid of frequencies.

Syntax

```
[H,wout] = freqresp(sys)  
H = freqresp(sys,w)  
H = freqresp(sys,w,units)  
[H,wout,covH] = freqresp(sys,___)
```

Input Arguments

sys – Dynamic system
dynamic system model | model array
w – Frequencies
vector
units – Units of the values in the input frequency vector
rad/TimeUnit (default) | 'cycles/TimeUnit' | 'rad/s' | ...

Output Arguments

H – Frequency response values
array
wout – Output frequencies corresponding to the frequency response
vector
covH – Covariance of the frequency response
array

Examples

Frequency Response of SISO System
Compute Frequency Response of System
Compute Frequency Response on Specified Frequency Grid
Compute Frequency Response and Associated Covariance

See also `evalfr`, `bode`, `nyquist`, `nichols`, `sigma`, `interp`,
`spectrum` (System Identification Toolbox)

Introduced in Control System Toolbox before R2006a
Documentation for `freqresp`
Other uses of `freqresp`

help `nyquist`

nyquist – Nyquist response of dynamic system
This MATLAB function returns the real and imaginary parts of the frequency response at each frequency in the vector `wout`.

Syntax

```
[re,im,wout] = nyquist(sys)  
[re,im,wout] = nyquist(sys,w)  
[re,im,wout,sdre,sdim] = nyquist(sys,w)  
  
nyquist(___)
```

Input Arguments

sys – Dynamic system
dynamic system model | model array
w – Frequencies
{wmin,wmax} | vector | []

Output Arguments

re – Real part of system response
3-D array
im – Imaginary part of system response
3-D array
wout – Frequencies
vector
sdre – Standard deviation of real part
3-D array | []
sdim – Standard deviation of imaginary part
3-D array | []

Examples

Nyquist Plot of Dynamic System
Nyquist Plot at Specified Frequencies
Nyquist Plot of Several Dynamic Systems
Nyquist Plot with Specified Line Attributes
Obtain Real and Imaginary Parts of Frequency Response
Nyquist Plot of MIMO System
Create Nyquist Plot of Identified Model with Response Uncertainty
Nyquist Plot of Model with Complex Coefficients

See also `nichols`, `sigma`, `bode`, `nyquistplot`

Introduced in Control System Toolbox before R2006a
Documentation for `nyquist`
Other uses of `nyquist`

help `bode`

bode – Bode frequency response of dynamic system
This MATLAB function computes the frequency response of dynamic system model `sys` and returns the magnitude and phase of the response at each frequency in the vector `wout`.

Syntax
[`mag`,`phase`,`wout`] = **bode**(`sys`)
[`mag`,`phase`,`wout`] = **bode**(`sys`,`w`)
[`mag`,`phase`,`wout`,`sdmag`,`sdphase`] = **bode**(`sys`,`w`)

bode(___)

Input Arguments
`sys` – Dynamic system
dynamic system model | model array
`w` – Frequencies
{`wmin`,`wmax`} | vector | []

Output Arguments
`mag` – Magnitude of system response
3-D array
`phase` – Phase of system response
3-D array
`wout` – Frequencies
vector
`sdmag` – Standard deviation of magnitude
3-D array | []
`sdphase` – Standard deviation of phase
3-D array | []

Examples
Bode Plot of Dynamic System
Bode Plot at Specified Frequencies
Compare Bode Plots of Several Dynamic Systems
Bode Plot with Specified Line Attributes
Obtain Magnitude and Phase Data
Magnitude and Phase of MIMO System
Bode Plot of Identified Model
Obtain Magnitude and Phase Standard Deviation Data of Identified Model
Bode Plot of Model with Complex Coefficients

See also `bodeplot`, `freqresp`, `nichols`, `nyquist`, `step`

Introduced in Control System Toolbox before R2006a
Documentation for `bode`
Other uses of `bode`

Moreover, the following custom functions are available:

`addpath(genpath("BodeDiagram"))`

help `asympt_bode`

```
*** asympt_bode(SISOmodel, om_vector) ***
[MagData,PhaseData,out_om_v] = asympt_bode(SISOmodel, om_vector)

Asymptotic Bode diagram of a transfer function.%
The SISOmodel can be any of the available LTI system models,
i.e. a tf object (preferred), an ss object or a zpk object.
In the event that SISOmodel is not a tf object, it is converted to tf.

om_vector --> angular frequency values used to draw the graphs
              (OPTIONAL – if not present, a suitable frequency range
              is estimated on the basis of zeros and poles. An
              appropriate number of values for the angular pulsation
              in the newly determined range is then chosen).

If MagData and PhaseData vectors are not requested in the output,
asymptotic modulus and phase graphs are plotted in graphs
on a semi-logarithmic scale.
The array out_om_v contains the angular frequency values
used to compute the asymptotic diagram of the frequency response.

Last Update: 2024/05/24
-----
```

help `drawBodediagrams`

drawBodediagrams()
Plotting Bode diagrams of the frequency response (asymptotic and actual superimposed diagrams) of continuous-time transfer functions.

INPUT variables:

Gs: transfer function, described as TF object; MANDATORY variable

omega_values: angular frequency values used to draw the graphs
(OPTIONAL – if not present, a suitable frequency range is estimated on the basis of zeros and poles. An appropriate number of values for the angular pulsation in the newly determined range is then chosen).

asBode_color: colour of asymptotic graphs
(array: three values between 0 and 1) – OPTIONAL
Default value: [1, 0, 0] <--> red

asBode_line_width: line thickness for asymptotic graphs – OPTIONAL
Default value: 2

asBode_line_style: line style for asymptotic diagrams
(see 'LineStyle' in plot()) – OPTIONAL
Default value: '-'

bode_color: colour of actual graphs
(array: three values between 0 and 1) – OPTIONAL
Default value: [0, 0, 1] <--> blue

bode_line_width: line thickness for actual graphs – OPTIONAL
Default value: 1.5

bode_line_style: line style for actual diagrams
(see 'LineStyle' in plot()) – OPTIONAL
Default value: '-'

fig_handler: The figure handler of a figure already in use, where Bode diagrams are drawn.
If it does not exist, a new figure is created.

comp_asympt_vs_actual_ph_diagsFLAG:
a logic flag indicating whether the 360-degree jump between the asymptotic phase diagram and the actual phase diagram should be compensated (if present).
Default value: true

OUTPUT variables (optional): the handlers of the couple of axes corresponding to the two subplots.

help drawPolarDiagramFreqResp

drawPolarDiagramFreqResp()
Plotting polar diagrams of the frequency response of continuous-time transfer functions.

INPUT variables:

Gs: transfer function, described as TF object; MANDATORY variable

omega_values: angular frequency values used to draw the graphs
(OPTIONAL – if not present, a suitable frequency range is estimated).

polar_color: colour of actual graphs
(array: three values between 0 and 1) – OPTIONAL
Default value: [0, 0, 1] <--> blue

polar_line_width: line thickness for actual graphs – OPTIONAL
Default value: 2.0

polar_line_style: line style for actual diagrams
(see 'LineStyle' in plot()) – OPTIONAL
Default value: '-'

markerSize: is the size (in pt.) of the circular markers used to put in evidence the points in the polar draw corresponding to null angular frequency and frequency going to infinity. If empty or zero, no markers will be drawn at all. – OPTIONAL
Default value: 6

om_selected: some angular frequency values at which you want to place a circular marker (black) in the graph. – OPTIONAL
Default value: empty array

fig_handler: The figure handler of a figure already in use, where the polar diagram has to be drawn.
If it does not exist, a new figure is created.

legendFLAG: a logical flag; if true an automatic legend will be inserted in the plot; otherwise, no legend will appear.

OUTPUT variable (optional): the handler of the axes corresponding to the plot.

Remark

Please, refer to the live script named [LTIfreqRespBODEDiagrams.mlx](#) for examples on how to use evalfr(), freqresp(), bode(), asympt_bode() and drawBodediagrams().

Polar Diagrams of the Frequency Response using MATLAB

Example 1: A Strictly Proper Transfer Function with Real Poles

Consider the LTI system described by the transfer function:

$$G(s) = \frac{10}{(1 + 10s)(1 + 2s)}, \quad \begin{cases} g = 0 & \mu = 10 \\ \tau_1 = 10 & \tau_2 = 2 \\ \omega_1 = 0.1 & \omega_2 = 0.5 \end{cases}$$

The polar diagram of the frequency response of $G(s)$ is

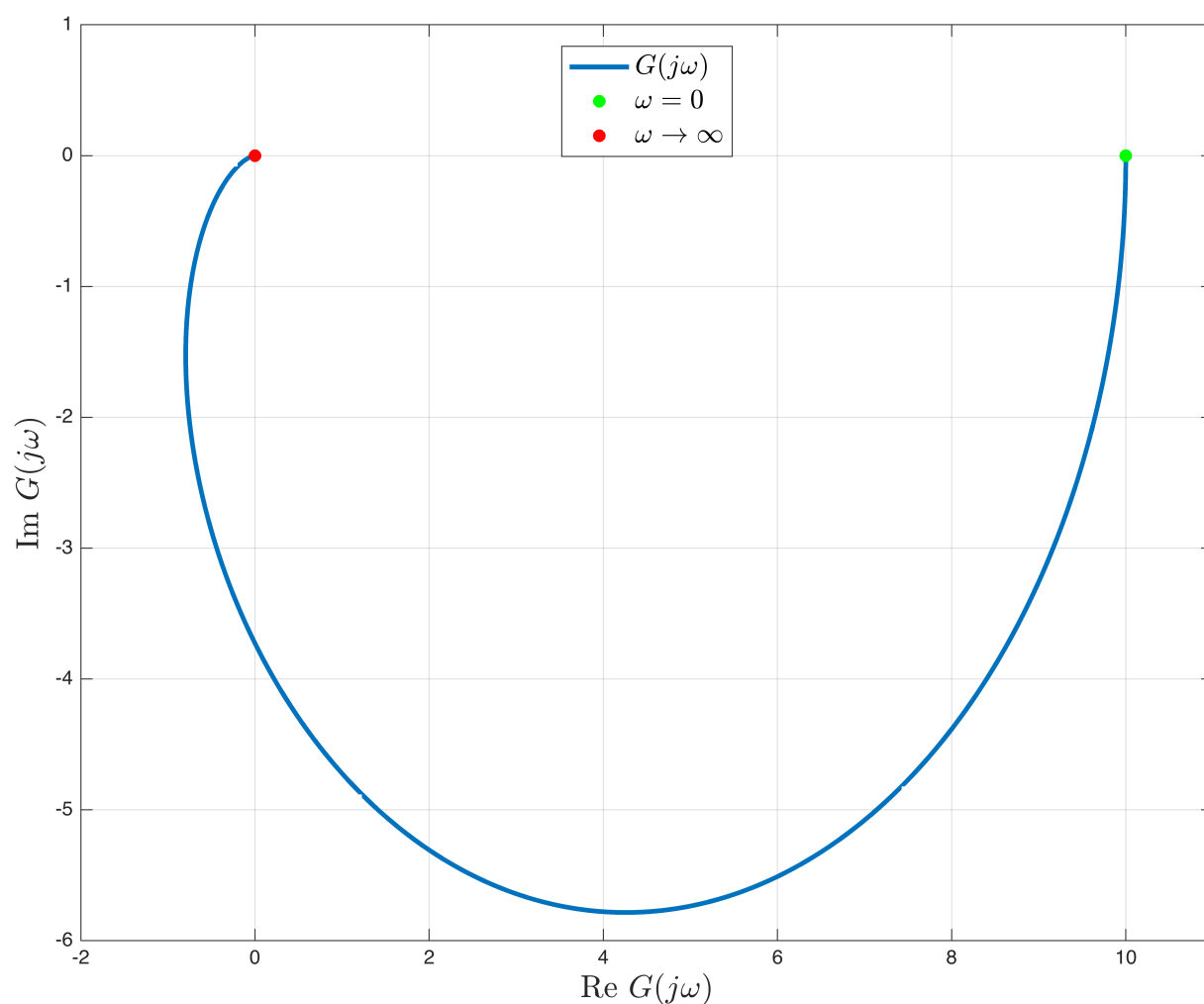
```
clear
s=tf('s');

Gs = 10/((1+10*s)*(1+2*s));

omVALS = logspace(-10,6, 1e4);
% We want to evaluate the frequency response at 10000
% angular pulsation values between 10^-10 and 10^6 rad/s.
%
% ** Note **:
%   The greater the number of angular pulsation values
%   used to evaluate the frequency response, the better
%   the accuracy of the diagram.

Bcolors = [0 0.4470 0.7410; ...
           0.9290 0.6940 0.1250; ...
           0.4940 0.1840 0.5560; ...
           0.4660 0.6740 0.1880; ...
           0.3010 0.7450 0.9330; ...
           0.8500 0.3250 0.0980; ...
           0.6350 0.0780 0.1840]; % some different colors
                                % for the actual diagrams

hf = figure('Units','centimeters','Position',[0.01, 0.01, 26, 20]);
drawPolarDiagramFreqResp(Gs, omVALS, Bcolors(1, :), 2.5, '-', 6, [], hf, true);
```



Example 2: A Couple of Complex Poles, varying the Damping Factor

Consider the LTI system described by the transfer function:

$$G(s) = \frac{25}{25 + 10\xi s + s^2}, \quad \xi = 0.9, 0.7, 0.5, 0.3, 0.1 \quad \omega_n = 5 \text{ rad/s}$$

```
clear

xiVAL = [0.1, 0.3, 0.5, 0.7, 0.9]; % the damping factor values
Nxi = numel(xiVAL); % how many damping factors?

Gs_xi = cell(Nxi, 1);
% let's preallocate a data-structure for storing the different transfer functions

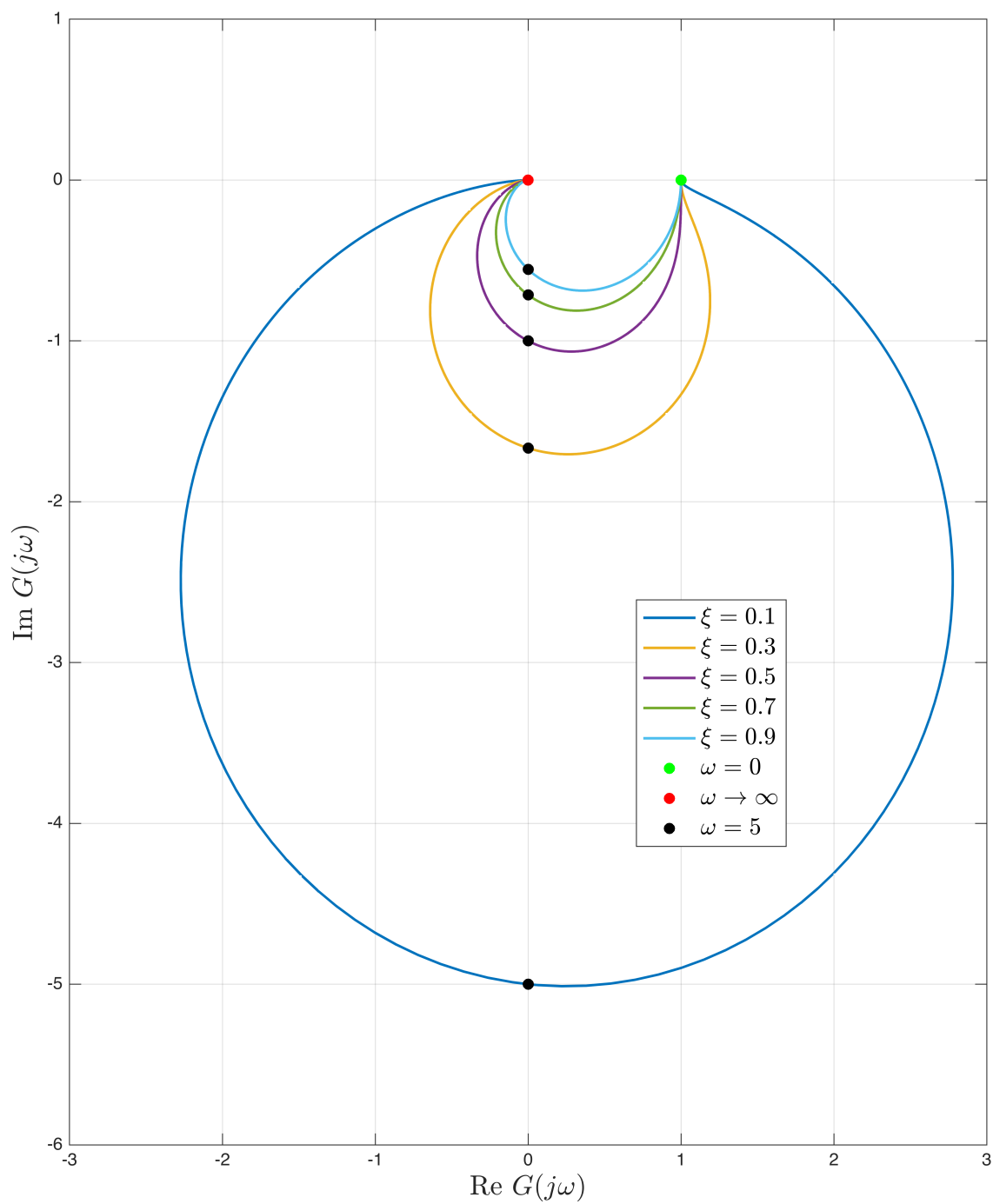
% generate the transfer functions, with different damping factors
numGs = 25; % the coeffs of the polynomial at the numerator in G(s)
for n=1:Nxi
    denGs = [ (1) (10*xiVAL(n)) 25];
    % the coeffs of the polynomial at the denominator in G(s),
    % in descending order
    Gs_xi{n} = tf(numGs, denGs); % let's define the transfer function G(s)
                                % and store it in the data-structure
end % for

omVALS = logspace(-10,4, 1e4);
% We want to evaluate the frequency response at 10000
% angular pulsation values between 10^-10 and 10^4 rad/s.
%
% ** Note **:
%   The greater the number of angular pulsation values
%   used to evaluate the frequency response, the better
%   the accuracy of the plot.

% let's create the diagrams: we need to store the handler of the figure
% refer to the help of drawPolarDiagramFreqResp( )

Bcolors = [0 0.4470 0.7410; ...
           0.9290 0.6940 0.1250; ...
           0.4940 0.1840 0.5560; ...
           0.4660 0.6740 0.1880; ...
           0.3010 0.7450 0.9330; ...
           0.8500 0.3250 0.0980; ...
           0.6350 0.0780 0.1840]; % some different colors
                                % for the actual diagrams

hf = figure('Units','centimeters','Position',[0.01, 0.01, 24, 28]);
%t = tiledlayout(2, 3); % let's create rooms for plots in a 3x3 grid
%nextile(1); % % let's move to tile 1
for n=1:Nxi
    hax = drawPolarDiagramFreqResp(Gs_xi{n}, omVALS, ...
                                   Bcolors(n, :), 1.5, '-', 6, 5, hf, false);
end % for
legend(hax, ['$\xi = ', num2str(xiVAL(1)),$'], '', '', '', ...
           ['$\xi = ', num2str(xiVAL(2)),$'], '', '', '', ...
           ['$\xi = ', num2str(xiVAL(3)),$'], '', '', '', ...
           ['$\xi = ', num2str(xiVAL(4)),$'], '', '', '', ...
           ['$\xi = ', num2str(xiVAL(5)),$'], '$\omega = 0$', '$\omega \to \infty$', ...
           '$\omega = 5$', 'Interpreter','latex', 'fontsize',14, 'Location', 'best');
```



Example 3: a Minimum-Phase LTI System with a Pole at the Origin

Consider the LTI system described by the transfer function:

$$G(s) = \frac{s+10}{s(s+1)} = 10 \frac{\left(1 + \frac{s}{10}\right)}{s(1+s)}$$

Analysing the transfer function, you get

$$\begin{cases} g = 1 \\ \mu = 10 \end{cases} \Rightarrow \mu_{\text{dB}} = 20 \text{ dB} \quad \begin{cases} z_1 = -10 \\ p_1 = 0 \\ p_2 = -1 \end{cases}$$

```
clear
s = tf('s'); % the transfer function building helper

Gs = (s+10)/(s * (1+s) );

omVALS = logspace(-2,3, 5e4); % <-- Modify this interval and see what the plot looks like.

% We want to evaluate the frequency response at 50000
% angular pulsation values between 10^-2 and 10^3 rad/s.
%
% ** Note **:
%   The greater the number of angular pulsation values
%   used to evaluate the frequency response, the better
%   the accuracy of the diagram.

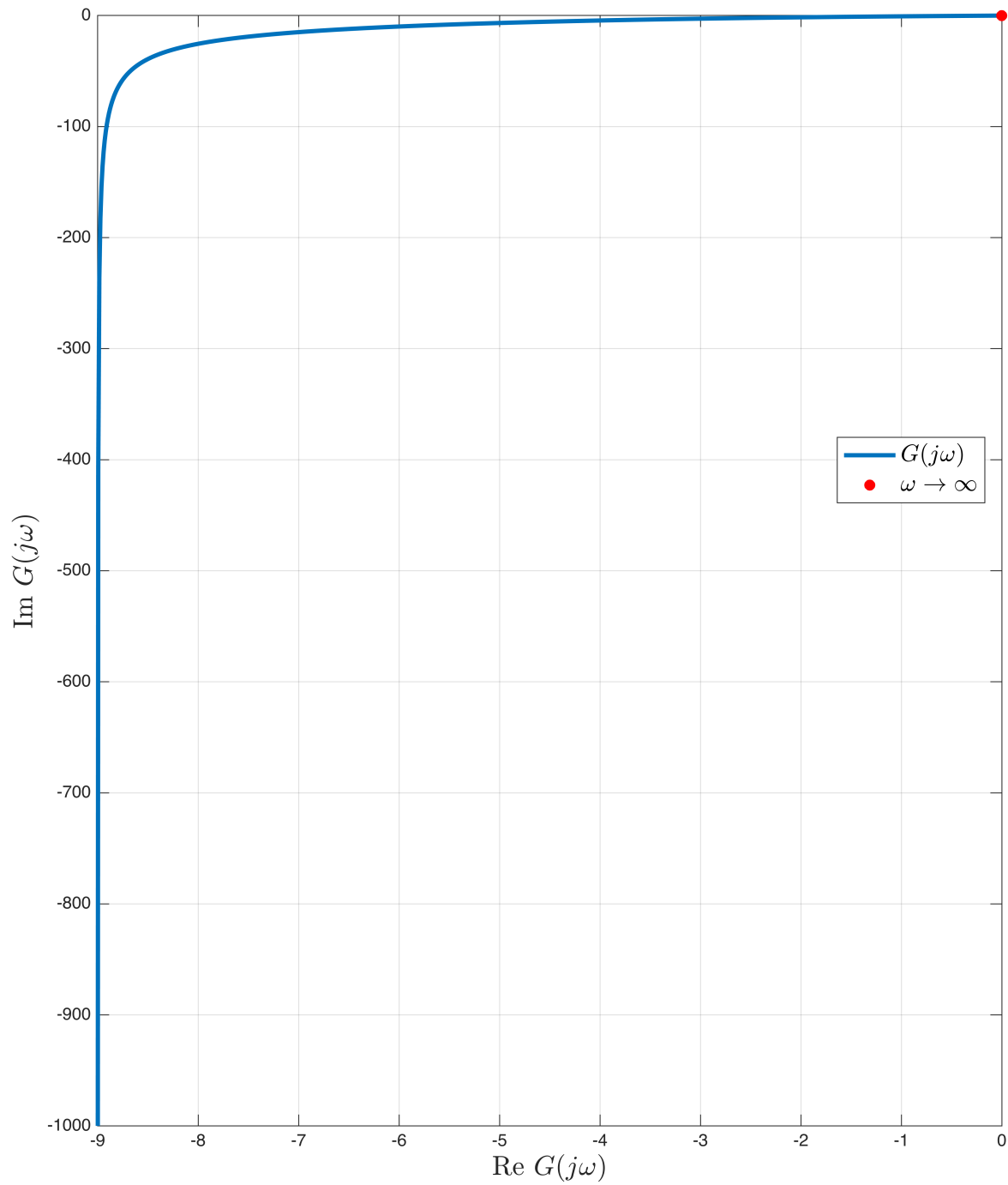
% let's create the diagrams: we need to store the handler of the figure
```

```
% refer to the help of drawPolarDiagramFreqResp( )

Bcolor = [0 0.4470 0.7410]; % a different color for the actual diagram

hf = figure('Units','centimeters','Position',[0.01, 0.01, 24, 28]);

hax = drawPolarDiagramFreqResp(Gs, omVALS, ...
                               Bcolor, 2.5, '-', 6, [], hf, false);
legend(hax, '$G(j \omega)$', '', '$\omega \to \infty$', ...
       'Interpreter','latex', 'fontsize',14, 'Location', 'best');
```



```
rmpath(genpath("BodeDiagram"))
```